



El concepto de herencia

La **herencia jerárquica** es un mecanismo que permite organizar clases de acuerdo a relaciones de **generalización-especialización**.

En la herencia jerárquica el diagrama de clases tiene la forma de un **árbol**

01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1

El concepto de polimorfismo

El **polimorfismo** en Ciencias de la Computación es el mecanismo que permite que un mismo nombre esté asociado a distintas abstracciones.

01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1

El polimorfismo en Java

Una **variable polimórfica** puede quedar asociada a objetos de diferentes clases.

Una **asignación polimórfica** liga un objeto de una clase a una variable declarada de otra clase.

Un **método polimórfico** incluye una o más variables polimórficas como parámetro.

Una **clase polimórfica** incluye uno o más métodos polimórficos.

01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1

La ligadura estática y la ligadura dinámica

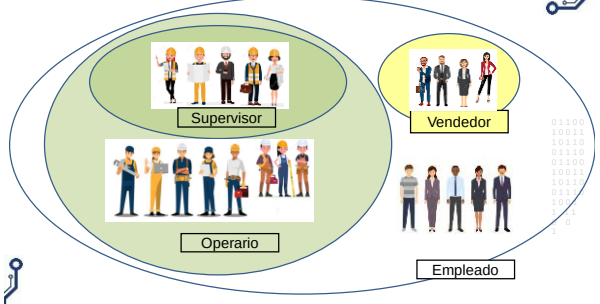
Las **ligaduras estáticas o vinculaciones estáticas** son las que se establecen antes de la ejecución.

Las **ligaduras dinámicas o vinculaciones dinámicas** son las que se establecen y pueden cambiar durante la ejecución.

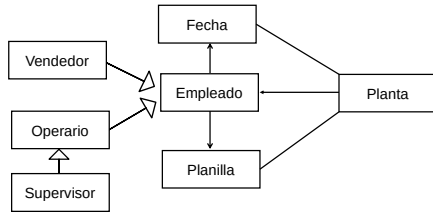
La **ligadura dinámica de código** es la vinculación en ejecución de un mensaje con un método.

01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1

Clasificación

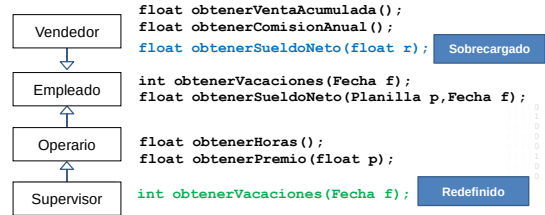


La relación entre clases



Todo objeto de una clase especializada ES UNA instancia de sus clases ancestro en la jerarquía de herencia.

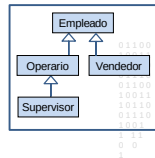
La sobrecarga y la redefinición



Las clases derivadas heredan los atributos y métodos de la clase Empleado

Las variables polimórficas

Empleado e1, e2, e3, e4;



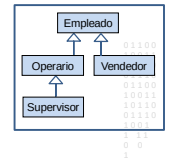
Las asignaciones polimórficas

Empleado e1, e2, e3, e4;

```

e1 = new Operario(125,21000,f);
e2 = new Supervisor(126,20000,f);
e3 = new Vendedor(120,30000,f,100000);
e4 = e2;
Operario e5 = new Supervisor(129,32000,f);
Supervisor e6 = new Supervisor(128,30000,f);

```



El tipo estático y el tipo dinámico

El **tipo estático** de una variable es el tipo que aparece en la declaración.

El **tipo dinámico** de una variable se determina en ejecución y corresponde a la clase del objeto referenciado.

El tipo estático y el tipo dinámico

Empleado e1, e2, e3, e4;

```

e1 = new Operario(125,21000,f);
e2 = new Supervisor(126,20000,f);
e3 = new Vendedor(120,30000,f,100000);
e4 = e2;
Operario e5 = new Supervisor(129,32000,f);
Supervisor e6 = new Supervisor(128,30000,f);

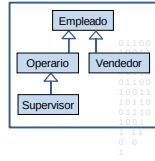
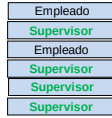
```

Variable	Tipo Estático	Tipo Dinámico
e1	Empleado	Operario
e2	Empleado	Supervisor
e3	Empleado	Vendedor
e4	Empleado	Supervisor
e5	Operario	Supervisor
e6	Supervisor	Supervisor

La ligadura dinámica

```
Empleado e1, e2, e3, e4;
e1 = new Operario(125,21000,f);
e2 = new Supervisor(126,20000,f);
e3 = new Vendedor(120,30000,f,100000);
e4 = e2;
Operario e5 = new Supervisor(129,32000,f);
Supervisor e6 = new Supervisor(128,30000,f);

int x = e1.obtenerVacaciones(h);
x = e2.obtenerVacaciones(h);
x = e3.obtenerVacaciones(h);
x = e4.obtenerVacaciones(h);
x = e5.obtenerVacaciones(h);
x = e6.obtenerVacaciones(h);
```



El tipo dinámico determina la ligadura entre un mensaje y un servicio

La sobrecarga y la redefinición

```
float obtenerVentaAcumulada();
float obtenerComisionAnual();
float obtenerSueldoNeto(float r);

int obtenerVacaciones(Fecha f);
float obtenerSueldoNeto(Planilla p, Fecha f);

float obtenerHoras();
float obtenerPremio(float p);

int obtenerVacaciones(Fecha f);
```

Cuando un objeto recibe un mensaje Java busca un método en la clase que corresponde al tipo dinámico, si no existe, busca en la clase padre y así siguiendo en la rama de ancestros hasta llegar a Object.

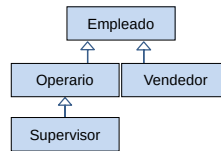
El chequeo de tipos en Java

En Java el chequeo de tipos es estático y establece restricciones sobre:

- las asignaciones polimórficas
- los mensajes que un objeto puede recibir

El polimorfismo es un mecanismo que favorece la productividad pero puede restringirse para brindar robustez.

El chequeo de tipos en Java



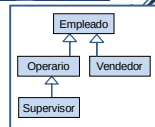
Tipo Estático	Tipos Dinámicos
Empleado	Empleado Operario Supervisor Vendedor
Operario	Operario Supervisor
Supervisor	Supervisor
Vendedor	Vendedor

El chequeo de tipos en Java

```
Empleado o1 = new Operario(125,21000,f);
Operario o2,o3;
```

```
o2 = o1;
o3 = new Empleado(126,20000,f);
```

Errores de compilación

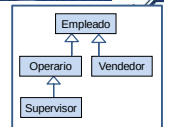


ERRORES DE COMPILACIÓN

```
Empleado o1 = new Operario(125,21000,f);
Operario o2 = new Operario(126,20000,f);
```

```
float p1 = o1.obtenerPremio(10);
float p2 = o2.obtenerPremio(10);
```

Error de compilación



El tipo estático restringe el polimorfismo en las asignaciones

El tipo estático determina los mensajes que el objeto puede recibir.

ERRORES DE COMPILACIÓN Y DE APLICACIÓN

```
Empleado v1 = new Vendedor(119,11000,£,10000);
Vendedor v2 = new Vendedor(111,10000,£,20000);
```

```
float sn1 = v1.obtenerSueldoNeto(p,h);
float sn2 = v1.obtenerSueldoNeto(r);
float sn3 = v2.obtenerSueldoNeto(p,h);
float sn4 = v2.obtenerSueldoNeto(r);
```

Possible error de aplicación

Error de compilación

Possible error de aplicación

Los errores de aplicación deben detectarse en la etapa de testing.

Producen resultados incorrectos pero no son advertidos por el compilador ni provocan la terminación anormal en ejecución.



Colección polimórfica

```
class Planta{
//Atributos de instancia
private Empleado[] t;
private int cant;
//Constructor
public Planta(int max){
/*Crea una colección con max elementos. Requiere max mayor a 0*/
t = new Empleado [max];
cant = 0;}
}
```

t es una estructura polimórfica, mantiene referencias a objetos de clase **Empleado** o de cualquiera de sus clases derivadas.

Colección polimórfica

```
public void alta (Empleado e){
/*asigna el empleado a la primera posición libre de la estructura.
Aumenta el valor de cant. Requiere que la clase cliente haya
verificado que la estructura no esté llena, e esté ligado y no haya un
empleado con el mismo número de legajo*/
t[cant] = e;
cant++;
}
```

La clase **Planta** es polimórfica porque define métodos polimórficos.

```
Planta pla = new Planta(5);
Operario o = new Operario(345,15000,£);
pla.alta(o);
```

Colección polimórfica

```
public void alta (Empleado e){
/*Asigna el empleado e a la primera posición libre de la estructura.
Aumenta el valor de cant. Requiere que la clase cliente haya
verificado que la estructura no esté llena, e esté ligado y no haya un
empleado con el mismo número de legajo*/
t[cant] = e;
cant++;
}
```

La clase **Planta** es polimórfica porque define métodos polimórficos.

```
Planta pla = new Planta(5);
Operario o = new Operario(345,15000,£);
pla.alta(o);
pla.alta(new Vendedor(346,20000,£,1000000));
pla.alta(new Vendedor(347,25000,£,50000));
```

Colección polimórfica

```
public Planta masVacaciones(int v, Fecha hoy){
/*Genera una colección con los empleados con más de v días
de vacaciones. Requiere hoy ligada*/
Planta nueva = new Planta(cantEmpleados());
for (int i = 0; i < cantEmpleados(); i++)
if (t[i].obtenerVacaciones(hoy) > v)
nueva.alta(t[i]);
return nueva;
}
```

La forma de computar la cantidad de días de vacaciones puede variar de un **Empleado** a otro, pero todas las definiciones del método **obtenerVacaciones** tienen la misma signatura.

Colección polimórfica

```
public int netoRango(float l1,float l2,Planilla p,
Fecha hoy){
/*Computa cuantos empleados tienen sueldos netos dentro
del rango l1 y l2. Requiere p y hoy ligadas.*/
int contador=0; float s;
for (int i = 0; i < cantEmpleados(); i++){
s = t[i].obtenerSueldoNeto(p,hoy);
if (l1 <= s && s <= l2)
contador++;
}
return contador;
}
```

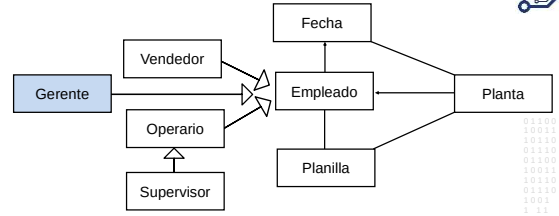
Possible error de aplicación, se ejecuta siempre el método definido en **Empleado**, aun cuando se trate de un vendedor.

Colección polimórfica

```
public int netoRango(float l1,float l2,Planilla p,
    Fecha hoy){
    /*Computa cuantos empleados tienen sueldos netos dentro
    del rango l1 y l2. Requiere p y hoy ligadas.*/
    int contador=0; float s;
    for (int i = 0; i < cantEmpleados(); i++){
        s = t[i].obtenerSueloNeto(1500);
        if (l1 <= s && s <= l2)
            contador++;
    }
    return contador;
}
```

Error de compilación

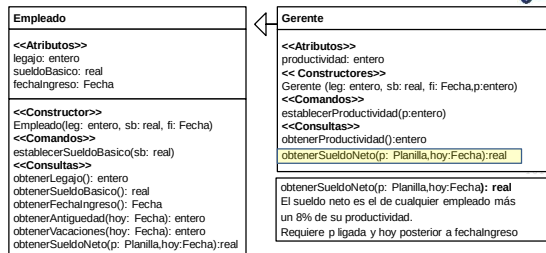
La herencia y la extensibilidad



Un cambio en el problema provoca un cambio en el diseño

La productividad aumenta cuando un cambio se resuelve agregando nuevas clases, sin modificar a las anteriores

La especialización de la clase Empleado



La clase Gerente es una clase derivada de la clase base Empleado

El impacto de los cambios

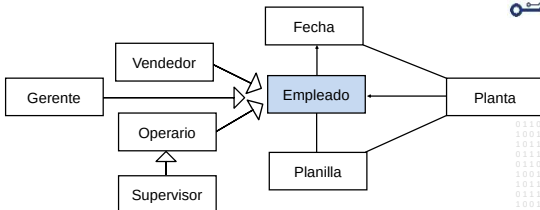
```
public float obtenerSueloNeto(Planilla p, Fecha hoy){
    /*El sueldo neto es el de cualquier empleado más
    un 8% de su productividad. Requiere p ligada y hoy posterior a
    fechaIngreso.*/
    return super.obtenerSueloNeto(p,hoy) + 0.08 * productividad;
}
```

El programador de la clase Gerente no necesita saber cómo se calcula el sueldo neto de cualquier empleado.

El cálculo del sueldo neto de un empleado puede cambiar y solo se modifica el código en la clase Empleado.

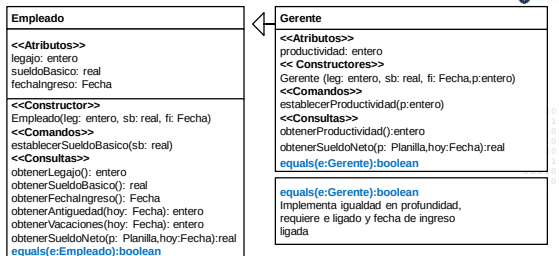
El cambio **no impacta** en la implementación de las clases derivadas porque usan **super**.

La herencia y la extensibilidad



Un cambio en la clase base no debería afectar a las clases derivadas, mientras la signatura y las responsabilidades no cambien

El método equals



El método **equals** está **sobrecargado** en clase Gerente

El método equals

```
public boolean equals (Empleado e) {
    /*Implementa igualdad en profundidad, requiere e ligado y fecha de
    ingreso ligada*/
    return legajo == e.obtenerLegajo() &&
        sueldoBasico == e.obtenerSueldoBasico() &&
        fechaIngreso.equals(e.obtenerFechaIngreso);
}

public boolean equals (Gerente e) {
    return super.equals(e) && productividad == e.obtenerProductividad();
}

Empleado e = new Empleado (100,15000,f);
Gerente g = new Gerente (100,15000,f, 500);
System.out.println (e.equals(g));
System.out.println (g.equals(e));
```

En la clase Empleado

En la clase Gerente

El mensaje **equals** se liga al método de la clase **Empleado**.

El método equals

```
public boolean equals (Empleado e) {
    /*Implementa igualdad en profundidad, requiere e ligado y fecha de
    ingreso ligada*/
    return legajo == e.obtenerLegajo() &&
        sueldoBasico == e.obtenerSueldoBasico() &&
        fechaIngreso.equals(e.obtenerFechaIngreso);
}

public boolean equals (Gerente e) {
    return super.equals(e) && productividad == e.obtenerProductividad();
}

Empleado e = new Gerente (100,15000,f, 600);
Gerente g = new Gerente (100,15000,f, 500);
System.out.println (e.equals(g));
System.out.println (g.equals(e));
```

En la clase Empleado

En la clase Gerente

El mensaje **equals** se liga al método de la clase **Empleado**.

El método equals

```
public boolean equals (Empleado e) {
    /*Implementa igualdad en profundidad, requiere e ligado y fecha de
    ingreso ligada*/
    return legajo == e.obtenerLegajo() &&
        sueldoBasico == e.obtenerSueldoBasico() &&
        fechaIngreso.equals(e.obtenerFechaIngreso);
}

public boolean equals (Gerente e) {
    return super.equals(e) && productividad == e.obtenerProductividad();
}

Gerente e = new Gerente (100,15000,f, 600);
Gerente g = new Gerente (100,15000,f, 500);
System.out.println (e.equals(g));
System.out.println (g.equals(e));
```

En la clase Empleado

En la clase Gerente

El mensaje **equals** se liga al método de la clase **Gerente**.

El método equals

```
Empleado
<<Atributos>>
legajo: entero
sueldoBasico: real
fechaIngreso: Fecha
<<Constructor>>
Empleado(leg: entero, sb: real, fi: Fecha)
<<Comandos>>
establecerSueldoBasico(sb: real)
<<Consultas>>
obtenerLegajo(): entero
obtenerSueldoBasico(): real
obtenerFechaIngreso(): Fecha
obtenerAntiguedad(hoy: Fecha): entero
obtenerVacaciones(hoy: Fecha): entero
obtenerSueldoNeto(p: Planilla,hoy:Fecha):real
equals(e:Empleado):boolean

Gerente
<<Atributos>>
productividad: entero
<<Constructores>>
Gerente (leg: entero, sb: real, fi: Fecha,p:entero)
<<Comandos>>
establecerProductividad(p:entero)
<<Consultas>>
obtenerProductividad():entero
obtenerSueldoNeto(p: Planilla,hoy:Fecha):real
equals(e:Empleado):boolean
```

El método **equals** está redefinido en clase **Gerente**

El método equals

```
public boolean equals (Empleado e) {
    /*Implementa igualdad en profundidad, requiere e ligado y fecha de
    ingreso ligada*/
    return legajo == e.obtenerLegajo() &&
        sueldoBasico == e.obtenerSueldoBasico() &&
        fechaIngreso.equals(e.obtenerFechaIngreso);
}

public boolean equals (Empleado e) {
    return super.equals(e) && productividad == e.obtenerProductividad();
}

Error de compilación
```

En la clase Empleado

En la clase Gerente

El método **obtenerProductividad()** no está definido para los objetos de clase **Empleado**, aunque sean instancias de **Gerente**.

Igualdad y equivalencia

- p1 y p2 tienen la misma **identidad** si mantienen referencias a un mismo objeto
- Dos objetos son **equivalentes en forma superficial** si tienen el mismo estado interno, comparando atributo a atributo.
- Dos objetos son **equivalentes en profundidad** si, comparando atributo a atributo, tienen los mismos valores en los atributos de instancia de tipo elemental y los atributos de instancia de tipo clase son equivalentes.

Igualdad y equivalencia

- El método **equals** implementa el servicio que permite decidir si dos objetos son equivalentes.
- El diseño especifica si la equivalencia debe implementarse en forma superficial o en profundidad.
- Si **equals** se implementa en forma superficial la comparación para decidir si tienen el mismo estado interno se realiza usando el operador relacional, de modo que se compara la identidad de los atributos de clases asociadas.
- Si **equals** se implementa en profundidad, la comparación sigue siendo atributo por atributo, pero los atributos de tipo clase se comparan usando el método **equals** de la clase asociada.

```
01100
10011
10110
01110
01100
10011
10110
01110
10011
01110
10011
1 11
0 0
0 0
1
```

Igualdad y equivalencia

- El diseño podría ser más flexible y especificar que dos objetos son equivalentes si coinciden en el valor de un atributo o de un subconjunto de atributos. Por ejemplo, dos vehículos podrían ser equivalentes si coinciden en la patente.
- La definición de igualdad puede estar ligada a un concepto específico. Por ejemplo, si modelamos un polígono como una secuencia de puntos, dos polígonos son iguales si mantienen el mismo conjunto de puntos y en el mismo orden, aunque la secuencia sea diferente en cada uno.

```
01100
10011
10110
01110
01100
10011
10110
01110
10011
01110
10011
1 11
0 0
0 0
1
```

Igualdad, equivalencia y herencia

- Dos objetos que son instancias de una clase C o de alguna de sus clases derivadas, son equivalentes en forma superficial si mantienen los mismos valores en los atributos compartidos, comparando atributo a atributo.
- Dos objetos que son instancias de una clase C o de alguna de sus clases derivadas, son equivalentes en forma superficial si tienen el mismo tipo dinámico y mantienen los mismos valores en todos los atributos, comparando atributo a atributo.

```
01100
10011
10110
01110
01100
10011
10110
01110
10011
01110
10011
1 11
0 0
0 0
1
```

Igualdad, equivalencia y herencia

```
public boolean equals (Empleado e) {
    /*Implementa igualdad en profundidad, requiere e ligado y fecha de
    ingreso ligada*/
    boolean igual=false;
    if (this == e) igual = true;
    else if (this.getClass() == e.getClass())
        igual = legajo == e.obtenerLegajo() &&
            sueldoBasico == e.obtenerSueldoBasico() &&
            fechaIngreso.equals(e.obtenerFechaIngreso());
    return igual;
}
```

En la clase Empleado

```
public boolean equals (Empleado e) {
    boolean igual=false;
    if (this == e) igual = true;
    else if (this.getClass() == e.getClass()){
        Gerente g = (Gerente) e;
        igual = super.equals(e) && productividad==g.obtenerProductividad();
    }
    return igual;
}
```

En la clase Gerente

Igualdad, equivalencia y herencia

Si se ejecuta el método **equals** de la clase **Gerente**, podemos asegurarle al compilador que el objeto que recibió el mensaje es de clase **Gerente** y, por lo tanto, con el **if** que usa **getClass()**, también podemos asegurar que el parámetro es de la misma clase.

El casting le "indica" al compilador que no debe realizar el chequeo de tipos, el programador asegura que el tipo dinámico de **e** es **Gerente**, es decir **e** mantiene una referencia a una instancia de **Gerente**.

```
public boolean equals (Empleado e) {
    boolean igual=false;
    if (this == e) igual = true;
    else if (this.getClass() == e.getClass()){
        Gerente g = (Gerente) e;
        igual = super.equals(e) && productividad==g.obtenerProductividad();
    }
    return igual;
}
```

En la clase Gerente

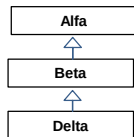
Igualdad, equivalencia y herencia

La asignación es válida porque el **casting** relaja el control del compilador.

```
public boolean equals (Empleado e) {
    boolean igual=false;
    if (this == e) igual = true;
    else if (this.getClass() == e.getClass()){
        Gerente g = (Gerente) e;
        igual = super.equals(e) && productividad==g.obtenerProductividad();
    }
    return igual;
}
```

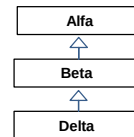
En la clase Gerente

Tipo Estático	Tipos Dinámicos
Alfa	Alfa Beta Delta
Beta	Beta Delta
Delta	Delta



```

Alfa v1 = new Alfa();
Alfa v2 = new Beta();
Alfa v3 = new Delta();
  
```



```

p(int x)
q(int x)
r(int x)
q(int y)
s(int y)
q(int z)
r(int z)
s(String z)
  
```

```

Alfa v1 = new Alfa();
Alfa v2 = new Beta();
Alfa v3 = new Delta();
Delta v4 = new Delta();
  
```



v1.p(1);	Alfa
v2.p(1);	Alfa
v3.p(1);	Alfa
v4.p(1);	Alfa

```

Alfa v1 = new Alfa();
Alfa v2 = new Beta();
Alfa v3 = new Delta();
  
```



v1.q(1);	Alfa
v2.q(1);	Beta
v3.q(1);	Delta

El **tipo dinámico** determina la **ligadura** entre el mensaje y el método.

```

Alfa v1 = new Alfa();
Alfa v2 = new Beta();
Alfa v3 = new Delta();
  
```



v1.r(1);	Alfa
v2.r(1);	Alfa
v3.r(1);	Delta

El **tipo dinámico** determina la **ligadura** entre el mensaje y el método.

```

Alfa v1 = new Alfa();
Beta v2 = new Beta();
Delta v3 = new Delta();
  
```

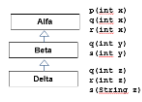


v2 = v1;	Error
v3 = v1;	Error
v3 = new Beta();	Error


```

Alfa v1 = new Beta();
Beta v2 = new Delta();
Beta v3 = new Delta();

```



```
v1.s(1);
```

Error

```
v2.s(1);
```

```
v3.s("abc");
```

Error